

Ali Abdullah Ahmad

20031246

CS 600

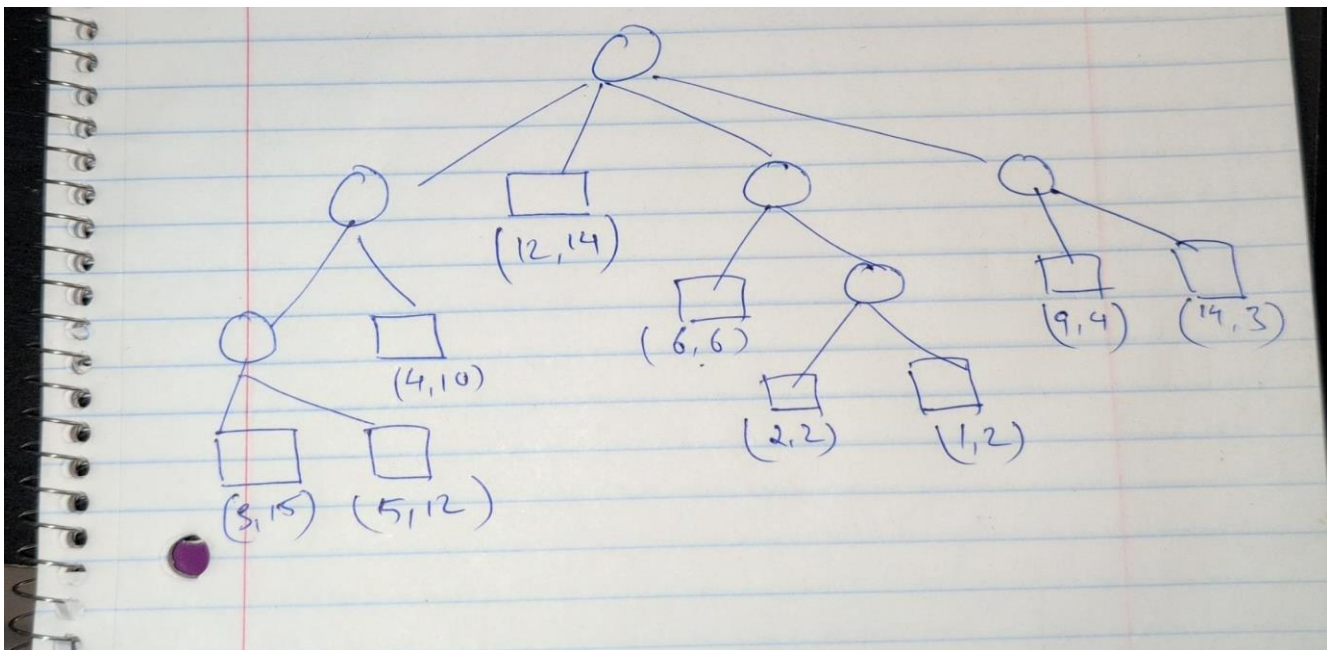
Q1. No. 21.5.7

(a) Draw a quadtree for the following set of points, assuming a 16×16 bounding box:

$\{(1, 2), (4, 10), (14, 3), (6, 6), (3, 15), (2, 2), (3, 12), (9, 4), (12, 14)\}$.

Ans.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1		a														
2		b														
3												d			f	
4										e						
5																
6						c										
7																
8																
9				h												
10																
11																
12														g		
13																
14			i													
15																
16																



Q2. No. 21.5.13

- (a) Design a static data structure (which does not support insertions and deletions) that stores a two-dimensional set S of n points and can answer, in $O(\log^2 n)$ time, queries of the form $\text{countAllInRange}(a, b, c, d)$, which return the number of points in S with x -coordinates in the range $[a, b]$ and y -coordinates in the range $[c, d]$. What is the space used by this structure?

Sol.

To design a static data structure that stores a two-dimensional set S of n points and answers range queries of the form $\text{countAllInRange}(a, b, c, d)$ in $O(\log^2 n)$ time, we use a 2D Range Tree.

Structure:

- Primary Tree (T_1):
A balanced binary search tree (e.g., AVL or Red-Black Tree) built on the x -coordinates of the points.
- Secondary Trees (T_2):
Each node in the primary tree stores a secondary balanced BST containing all points in its subtree, sorted by y -coordinate.

Query – $\text{countAllInRange}(a, b, c, d)$:

1. Use the primary tree to find all canonical subsets of points whose x -coordinates fall within the range $[a, b]$. This takes $O(\log n)$ time and returns $O(\log n)$ nodes.
2. For each of these $O(\log n)$ nodes, query the associated secondary tree to count points with y -coordinates in the range $[c, d]$. Each such query takes $O(\log n)$ time.
3. Thus, the total query time is $O(\log n \cdot \log n) = O(\log^2 n)$

Space Complexity:

- Each node in the primary tree maintains a secondary tree containing $O(n)$ points across all levels.
- Therefore, the total space required is $O(n \log n)$.

Conclusion:

- Data Structure: 2D Range Tree
- Query Time: $O(\log^2 n)$
- Space Used: $O(n \log n)$
- Insertions/Deletions: Not supported (static structure)

This structure efficiently supports 2D range counting queries in a static setting.

Q3. No. 21.5.27

- (a) Consider the previous exercise, but instead of explaining how to use D to answer box-containment range queries, now explain how to build an efficient data structure, D , that can store four-dimensional range queries for four-dimensional points in $O(s + \log^3 n)$ time, where s is the number of answers. What is the space complexity of your data structure, D ?

Ans.

To build an efficient data structure D for answering four-dimensional orthogonal range queries on n four-dimensional points with query time $O(s + \log^3 n)$, where s is the number of reported points, we use a range tree with fractional cascading and appropriate balancing.

Construction:

1. Primary Tree (T_1):
Build a balanced binary search tree (BST) on the first coordinate (x_1).
 - Each node in T_1 stores a secondary structure for the remaining three coordinates.
2. Secondary Tree (T_2):
For each node in T_1 , build another BST on the second coordinate (x_2) of the points stored at that node.
 - Again, each node in T_2 stores a tertiary structure for the remaining two dimensions.
3. Tertiary Tree (T_3):
At each node of T_2 , build a BST on the third coordinate (x_3), where each node stores a quaternary structure.
4. Quaternary Tree (T_4):
At each node of T_3 , build a sorted list of points based on the fourth coordinate (x_4).
 - Use fractional cascading on these lists to reduce the time to search along the path.

Query:

- At each level, the query time is logarithmic in n , with the top three levels contributing $O(\log^3 n)$ total time.
- The final level (x_4) uses fractional cascading to efficiently search the sorted list and retrieve all s matching points.

Time Complexity:

- Query Time: $O(s + \log^3 n)$
- Construction Time: $O(n \log^3 n)$

Space Complexity:

- Each level of the range tree adds a logarithmic factor in space.
- Total Space: $O(n \log^3 n)$

Thus, data structure D supports four-dimensional range queries in $O(s + \log^3 n)$ time using $O(n \log^3 n)$ space.

Q4. No. 22.6.7

Give a pseudocode description of the plane-sweep algorithm for finding a closest pair of points among a set of n points in the plane.

Ans.

Input:

A set P of n points in the 2D plane.

Output:

The closest pair of points and their Euclidean distance.

Pseudocode:

ClosestPair(P):

1. Sort the points in P by x -coordinate $\rightarrow P_x$
2. Initialize an empty balanced binary search tree T (ordered by y -coordinate)
3. Set $\text{minDist} \leftarrow \infty$
4. Initialize $\text{left} \leftarrow 0$
5. for i from 0 to $n - 1$:
 - a. Let $p \leftarrow P_x[i]$
 - b. While $(P_x[i].x - P_x[\text{left}].x) > \text{minDist}$:
 - Remove $P_x[\text{left}]$ from T
 - Increment left
 - c. Let $\text{Yrange} \leftarrow$ all points q in T such that $|q.y - p.y| < \text{minDist}$
 - d. For each point q in Yrange :
 - Compute distance d between p and q
 - If $d < \text{minDist}$:
 - $\text{minDist} \leftarrow d$
 - $\text{closestPair} \leftarrow (p, q)$
 - e. Insert p into T
6. Return closestPair and minDist

Explanation:

- The algorithm sweeps a vertical line from left to right across the sorted points.
- It maintains a balanced BST of candidate points sorted by y -coordinate.
- For each point, it removes points too far on the left (more than minDist away in x -axis).
- Then it only compares the point with others within a vertical strip of width $2 * \text{minDist}$, reducing unnecessary comparisons.

Time Complexity:

- **$O(n \log n)$** due to sorting and balanced BST operations.

Q5. No. 22.6.16

- (a) Let C be a collection of n horizontal and vertical line segments. Describe an $O(n \log n)$ -time algorithm for determining whether the segments in C form a simple polygon.

Ans.

Let C be a collection of n horizontal and vertical line segments.

Step 1: Sweep Line Algorithm for Intersections

- Use a plane sweep algorithm to detect all intersections between segments.
- Sort all segment endpoints by x-coordinate.
- Use a balanced BST to maintain the set of active segments.
- For each event point, check for intersections between vertical and horizontal segments.
- If any intersection occurs other than at endpoints, return False – the polygon is not simple.

Step 2: Build Adjacency Graph of Endpoints

- For each segment, insert its two endpoints into a dictionary or map.
- Each endpoint should appear in exactly two segments (i.e., have degree 2) for a simple polygon.
- If any endpoint has degree not equal to 2, return False.

Step 3: Verify a Single Closed Loop

- Use DFS or BFS to traverse the graph from any starting point.
- Ensure that:
 - All points are visited exactly once.
 - The traversal returns to the starting point, forming a single simple cycle.

Time Complexity:

- Plane sweep: $O(n \log n)$
- Graph construction: $O(n)$
- DFS traversal: $O(n)$
- Total time: $O(n \log n)$

Q6. No. 22.6.30

- (a) In machine learning applications, we often have some kind of condition defined over a set, S , of n points, which we would like to characterize—that is, "learn"—using some simple rule. For instance, these points could correspond to biological attributes of n medical patients and the condition could be whether a given patient tests positive for a given disease or not, and we would like to learn the correlation between these attributes and this disease. Think of the points with positive tests as painted "red," and the tests with negative tests as painted "blue." Suppose that we have a simple two-factor set of attributes; hence, we can view each patient as a two-dimensional point in the plane, which is colored as either red or blue. An ideal characterization, from a machine learning perspective, is if we can separate the points of S by a line, L , such that all the points on one side of L are red and all the points on the other side of L are blue. So suppose you are given a set, S , of n red and blue points in the plane. Describe an efficient algorithm for determining whether there is a line, L , that separates the red and blue points in S . What is the running time of your algorithm?

Ans.

Steps:

1. Partition the Set:
 - Separate the set S of n points into two sets:
 - R: Red points
 - B: Blue points
2. Construct Convex Hulls:
 - Compute the convex hull of the red points $\rightarrow CH(R)$
 - Compute the convex hull of the blue points $\rightarrow CH(B)$
 - This can be done using Graham's scan or Andrew's monotone chain, both in $O(n \log n)$ time.
3. Check for Intersection:
 - Check whether the two convex hulls intersect.
 - If $CH(R)$ and $CH(B)$ do not intersect, then there exists a line L that separates the two sets (by the separating hyperplane theorem).
 - If they do intersect, then no such line exists.

Conclusion:

- If the convex hulls do not intersect, then the red and blue points are linearly separable.
- If the convex hulls intersect, then there is no line that can separate them.

Time Complexity:

- Constructing each convex hull: $O(n \log n)$
- Intersection check: $O(n)$ (can be done in linear time if hulls are sorted)
- Total Time: $O(n \log n)$